Cross-Validation

2026-04-17

Introduction

Cross-validation is the primary tool for estimating how well a predictive model will generalise to unseen data. The in-sample error – the error of the fitted model on the data used to train it – is almost always too optimistic. Cross-validation repeatedly splits the data into training and held-out portions, fits the model on the training portion, evaluates on the held-out portion, and averages the held-out performance as an estimate of true generalisation error.

Prerequisites

The reader should understand the concepts of overfitting, bias and variance, and the training vs. test-set distinction. Familiarity with `tidymodels` is helpful but not assumed.

Theory

A model's **generalisation error** is its expected loss on a new observation drawn from the same population as the training data. The **training error** is its loss on the training data itself. The difference between them – the **optimism** of the training error – grows with model flexibility and shrinks with sample size.

k-fold cross-validation

The data are partitioned into k approximately equal folds. For each fold $j = 1, \ldots, k$, the model is fitted on the other $k - 1$ folds and evaluated on the held-out j-th fold. The k held-out losses are averaged. A common choice is $k = 5$ or $k = 10$, which balances bias (too few folds means each training set is noticeably smaller than the full data) against variance (too many folds means the estimates are correlated).

Leave-one-out cross-validation (LOOCV)

The extreme case $k = n$: each fold has exactly one observation. LOOCV has low bias (the training sets are nearly the full data) but high variance and high computational cost. It is the right choice when n is small and the model is fast to fit.

Stratified cross-validation

For classification with class imbalance, folds are constructed to preserve class proportions. This reduces variability in the estimated performance and avoids folds with zero minority-class examples.

Repeated k-fold

k-fold CV is repeated with different random partitions, and the results averaged. This reduces Monte Carlo variability in the estimate without the computational cost of LOOCV.

Nested cross-validation

When hyperparameters are tuned on CV, the tuned performance estimate is optimistically biased because the hyperparameters have been chosen to maximise the same metric. Nested CV uses an outer loop for honest

generalisation estimation and an inner loop for tuning: the outer folds never see the data used to choose the hyperparameters.

Assumptions

CV is a general technique, but it assumes:

1. The splits are representative of the data-generating process – the training and test folds come from the same distribution.
2. The observations are exchangeable (or the CV scheme respects the dependence structure – see below).
3. The performance metric is well-estimated by an average over folds.

For time-series or clustered data, standard CV leaks information between folds and overstates performance. Use time-series CV (expanding window) or group-blocked CV (by subject, site, etc.) in those cases.

R Implementation

```r
library(tidymodels)

data(penguins, package = "palmerpenguins")
penguins <- tidyr::drop_na(penguins)

set.seed(2026)
split <- initial_split(penguins, prop = 0.75, strata = species)
train <- training(split)
test  <- testing(split)

folds <- vfold_cv(train, v = 10, strata = species)

rec <- recipe(species ~ bill_length_mm + bill_depth_mm +
              flipper_length_mm + body_mass_g, data = train) |>
  step_normalize(all_numeric_predictors())

spec <- multinom_reg() |> set_engine("nnet")

wf <- workflow() |> add_recipe(rec) |> add_model(spec)

cv_results <- fit_resamples(wf, resamples = folds,
                            metrics = metric_set(accuracy, roc_auc))

collect_metrics(cv_results)
```

This performs stratified 10-fold CV for a multinomial logistic regression on the `penguins` dataset. `fit_resamples()` handles the CV loop; `collect_metrics()` summarises the fold-level performance.

Output & Results

A typical output:

.metric	.estimator	mean	n	std_err
accuracy	multiclass	0.981	10	0.008
roc_auc	hand_till	0.999	10	0.000

The mean across folds is the point estimate; the standard error is the Monte Carlo uncertainty across folds.

Interpretation

The 10-fold cross-validated accuracy is 0.981 (SE 0.008), and the ROC AUC is 0.999 (SE < 0.001). These are estimates of out-of-sample performance on the penguin species classification task. The held-out test set is still unused; it is reserved for the final, honest evaluation after all modelling decisions are locked.

Practical Tips

- Preprocessing (scaling, imputation, encoding) must go *inside* the resampling loop to prevent leakage. `recipes` with `workflow()` does this correctly by default.
- Report both the mean and the standard error across folds so readers know how confident the estimate is.
- For hyperparameter tuning, use nested CV (or a train/validation/test split) – tuning on the same CV you report is biased.
- For time series, use `rolling_origin()` or `sliding_window()` in `rsample`; never use `vfold_cv()` on serial data.
- Small datasets have high-variance CV estimates – consider reporting a 95% CI across folds or using repeated CV.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis